
I ARE
PROGRAMMER
I MAKE COMPUTER
BEEP BOOP BEEP
BEEP BOOP

Глава 2. ООП

Ежелев Г. И.

19 сентября 2026 г.



Материалы курса можно найти по ссылке:
https://github.com/haaroner/ezh239

Пространства имён

- Все имена всех объектов сгруппированы в пространства имён (namespace). Хорошей аналогией может являться файловая система, где каждая папка - пространство имён, а файл - название объекта.
- Вы можете добавлять свои namespace, чтобы избежать конфликта имён. Это может улучшить читаемость кода.

```
namespace Foo {
    void f() {
        std::cout << "Foo" << std::endl;
    }
}

namespace Boo {
    void f() {
        std::cout << "Boo" << std::endl;
    }
}
```

```
int main() {
    Foo::f();
    Boo::f();

    return 0;
}
```

```
using namespace Foo;
int main() {
    f();
    Boo::f();

    return 0;
}
```

база база база

- ООП - Объектно Ориентированное Программирование, т.е. мы смотрим на программу как на набор объектов, которые взаимодействуют между собой. Зачем это нужно?
- Мы постоянно хотим делать код более абстрактным, т.е.:
  что-то однотипное $\xrightarrow[\text{объединение}]{}$универсальное:
  - Несколько переменных $\rightarrow$ массив
  - Повторяющиеся действия $\rightarrow$ функция и т.п. Это принцип DRY - Dont Repeat Yourself

Если повторяющееся поведение сложное: есть и данные(переменные) и действия (функции) - их логично сгруппировать воедино.
Для этого и нужны классы.

ООП. Примеры

- Примеров с домиками и чертежами не будет
- Вращение мотора:
  •Данные: у каждого мотора есть два пина управления, текущая скорость, значение энкодера
  •Действия: Задать новую скорость, заблокировать мотор, узнать значение энкодера
  •Вы это обычно реализовывали 4-мя функциями void m1_run(speed), m2_run(speed), ...
  •Логично объеденить всё это, один раз описав логику взаимодействия с мотором и далее просто указав на каких пинах эти моторы есть.

Классы

А что такое объект?

- Объект - это область памяти, которая имеет: тип, размер и значение.
- Тип определяет, как этот объект себя ведёт. Пример типа: int, byte, char, string,...
- Т.е. объект - это данные + поведение - то, что нам и нужно.
- Если мы хотим хранить данные и работать с ними по-своему - нам нужен свой тип данных.
  - В примере с мотором: скорость - это число, но с ограничением (например не бывает скорости 110%). Также хочется, чтобы скорость менялась плавно (т.е. объект мотора сам следит за ускорением).

- Можно решить это через функцию set_speed и несколько переменных. Тогда это сработает с одним мотором.
- Если описать эту же логику внутри собственного (нового) типа данных - мы сможем применять её сразу ко всем 4-м моторам.
- То есть просто создадим 4 объекта с типом мотор.
- Класс - это собственный тип данных. У него есть свои данные (поля) + функции (методы),т.е. поведение.
  Класс описывает этот тип данных. Когда описание готово, мы можем создавать объекты этого класса (как мы создаем объекты класса string - переменные. Просто вместо string наш собственный тип данных).
- У string есть данные: массив символов (char) и набор методов: (.length,...)

Классы и объекты

ИТМО

- **Класс** задает какие характеристики есть у объекта
  - с помощью переменных (полей)



```
class Cat
name
age
color
owner
```

```
объект класса Cat
name  = Мурка
age   = 5.1
color = рыжий
owner = сосед
```



- Каждый **объект** имеет свои значения характеристик
  - значения полей задают состояние объекта



Зачем классы?

- Масштабирование:
  10 одинаковых датчиков $\Longrightarrow$ 1 класс и 10 одинаковых объектов, а не 10 одинаковых переменных, 10 одинаковых функций инициализации и т.п. (меньше кода)
- Разделение на уровни:
  •Каждый класс скрывает, как он работает внутри. Остальная программа может только взаимодействовать с его методами. Это позволяет разделить программу на уровни.
  •Например классы датчиков -> классы обработки данных -> класс поведения. Каждый уровень не парится, как работает уровень ниже, а просто использует метод ".get_data()".
  •Это оооочень удобно - когда вы пишете класс, вам нужно думать, только про этот маленький кусочек, а не про всю программу сразу. Получается модульность.

- Таким образом: если логика управления мотором изменится - нужно будет менять только код Motor, а moveRobot - нет (ведь moveRobot не думает, как устроен Motor, он только отдает команду motor.set_speed)

NB Наши новые объекты можно передавать куда-то (как вы передаете int и т.п.)
Например мы можем создать функцию moveRobot(int angle, int speed, motor m1, motor m2, motor m3, motor m4), которая будет принимать 4 объекта моторов и через них управлять движением робота

Создание классов

Класс состоит из:

- Конструктор - метод, вызывающийся при создании нового объекта. Название конструктора совпадает с названием класса.
- Данные - переменные.
- Методы - функции класса.
- Модификаторы доступа - определяющие, кто может использовать данные/методы public - кто угодно, private - только изнутри класса, protected - (?)
- Например внешним классам нельзя самим опрашивать энкодер, это будет делать сам класс Мотора. А вот прочитать переменную, содержащую положение колеса другие куски программы могут.

- См. пример для мотора в файле Prog_01.cpp
- А как объект выглядит в памяти? Разберем на примере класса - Rectangle - геометрическая фигура. (Пример на Java - детали могут отличаться от C++)

Объекты в памяти

ИТМО

```
Rectangle r1;
r1 = new Rectangle();
r1.width = 10.5;
r1.height = 8.2;
Rectangle r2;
r2 = r1;
r1 = new Rectangle();
r2 = null;
```





Объекты в памяти

ИТМО

```
Rectangle r1;
r1 = new Rectangle();
r1.width = 10.5;
r1.height = 8.2;
Rectangle r2;
r2 = r1;
r1 = new Rectangle();
r2 = null;
```

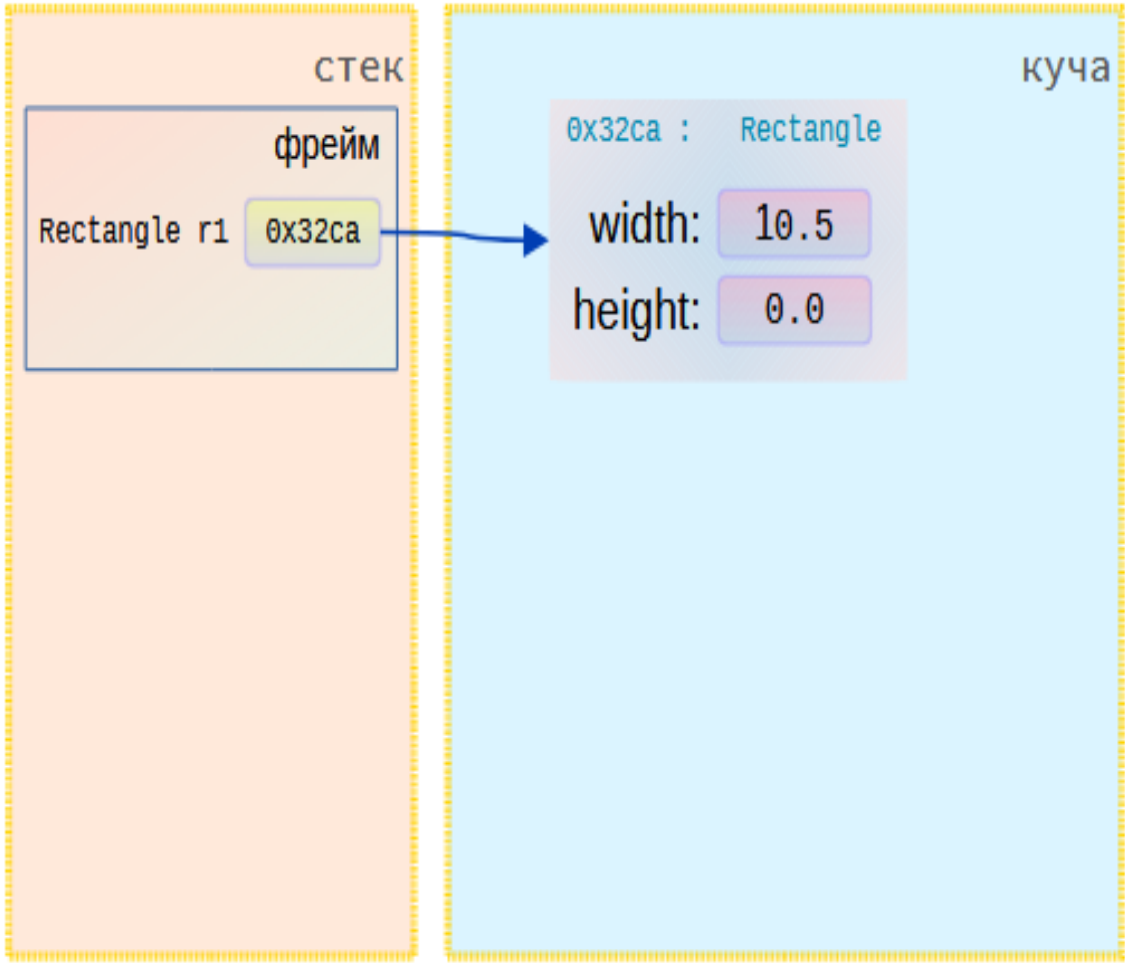




Объекты в памяти

ИТМО

```
Rectangle r1;
r1 = new Rectangle();
r1.width = 10.5;
r1.height = 8.2;
Rectangle r2;
r2 = r1;
r1 = new Rectangle();
r2 = null;
```

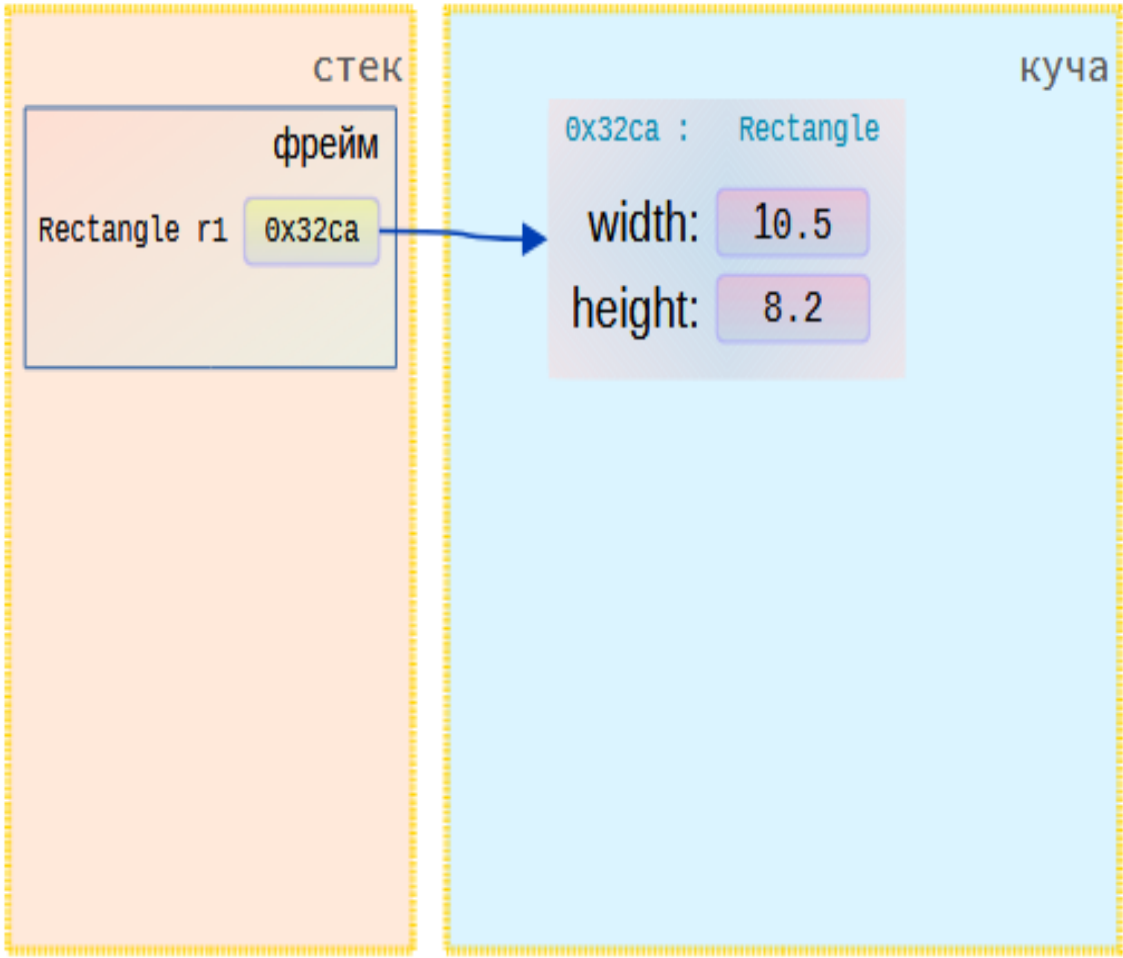




Объекты в памяти

ИТМО

```
Rectangle r1;
r1 = new Rectangle();
r1.width = 10.5;
r1.height = 8.2;
Rectangle r2;
r2 = r1;
r1 = new Rectangle();
r2 = null;
```

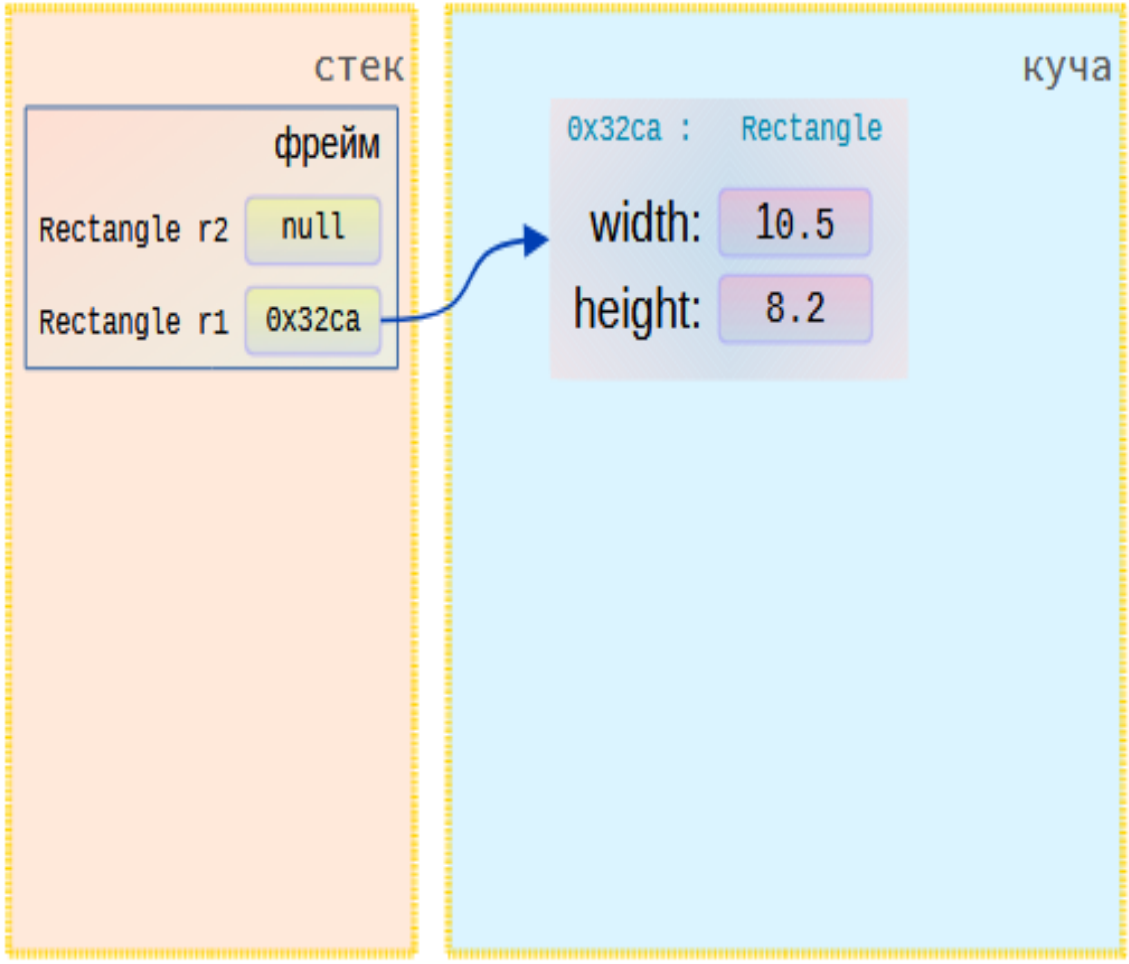




Объекты в памяти

ИТМО

```
Rectangle r1;
r1 = new Rectangle();
r1.width = 10.5;
r1.height = 8.2;
Rectangle r2;
r2 = r1;
r1 = new Rectangle();
r2 = null;
```





Объекты в памяти

ИТМО

```
Rectangle r1;
r1 = new Rectangle();
r1.width = 10.5;
r1.height = 8.2;
Rectangle r2;
r2 = r1;
r1 = new Rectangle();
r2 = null;
```

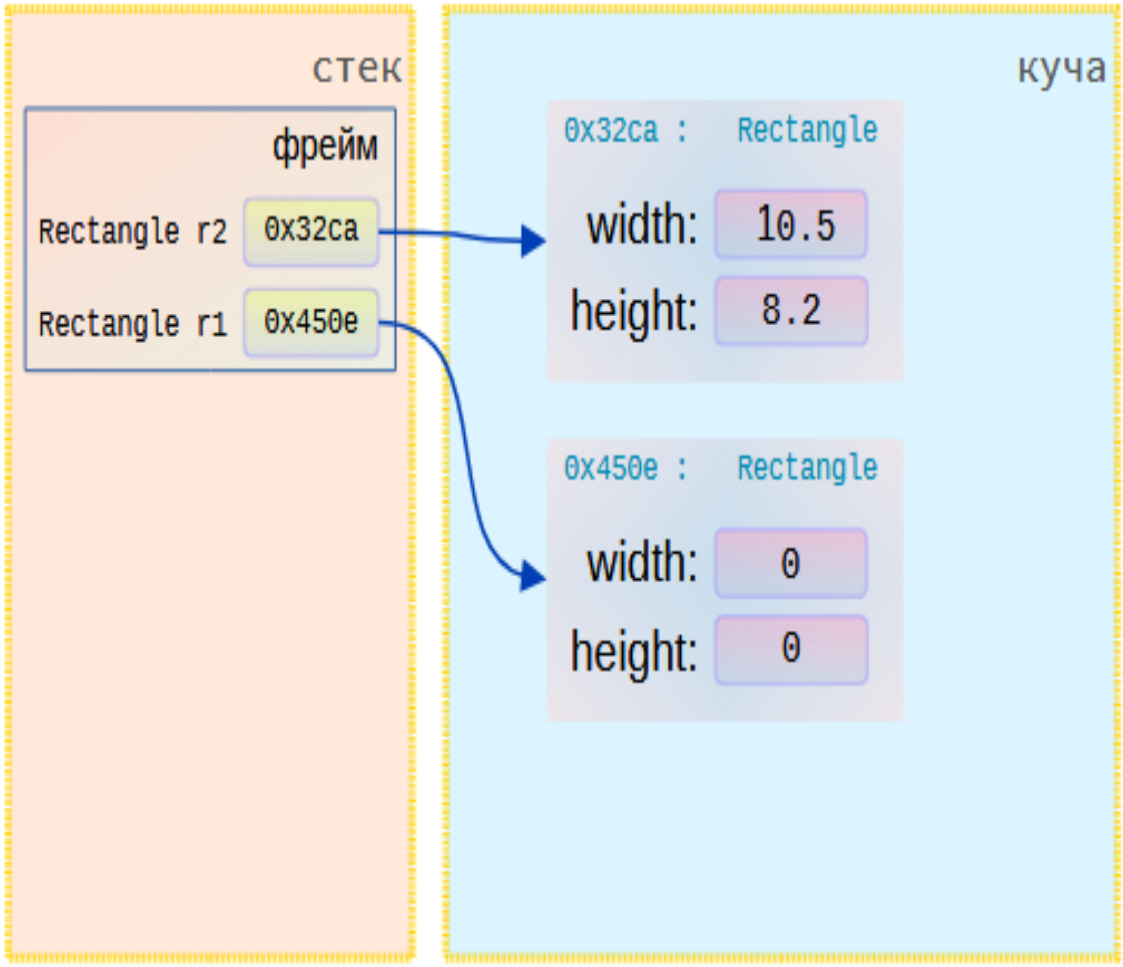




Объекты в памяти

ИТМО

```
Rectangle r1;
r1 = new Rectangle();
r1.width = 10.5;
r1.height = 8.2;
Rectangle r2;
r2 = r1;
r1 = new Rectangle();
r2 = null;
```

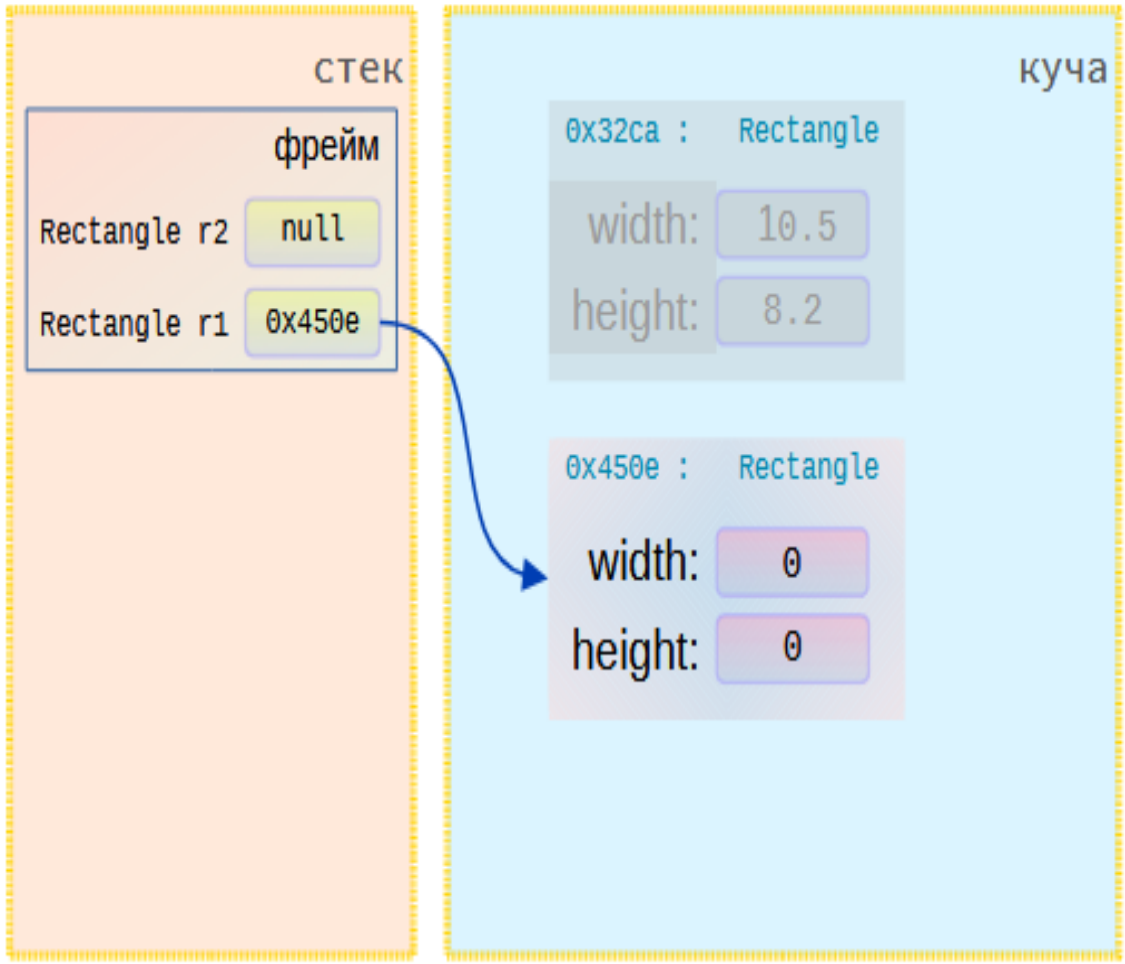




Объекты в памяти

ИТМО

```
Rectangle r1;
r1 = new Rectangle();
r1.width = 10.5;
r1.height = 8.2;
Rectangle r2;
r2 = r1;
r1 = new Rectangle();
r2 = null;
```





- Как вы заметили - ссылки на объекты тоже хранятся в стеке (тот самый стек из прошлой главы), а сами их данные нет.
- Данные и методы относятся к конкретному объекту
- Можно также сделать методы и данные, относящиеся ко всему классу (т.е. одна переменная или метод един(а) для всех объектов сразу).
- Это статический тип - static.
- Если у вас есть что-то внутри и вне класса с одинаковым именем - используйте ссылку this.(метод/переменная объекта). Эта ссылка неявно передаётся каждому методу
- Класс - кастомный тип данных (как int, char, string и т.п.).

Поля объекта и поля класса

ИТМО

- Объект (instance) имеет собственные значения полей
  - Для разных объектов значения полей разные
  - Это поля экземпляра (не статические)
- Особый вид поля - статическое (static field)
  - Статическое поле - общее для всех объектов класса
  - Его значение - одно для всего класса
  - Статическое поле - поле класса (в Metaspace - часть кучи)

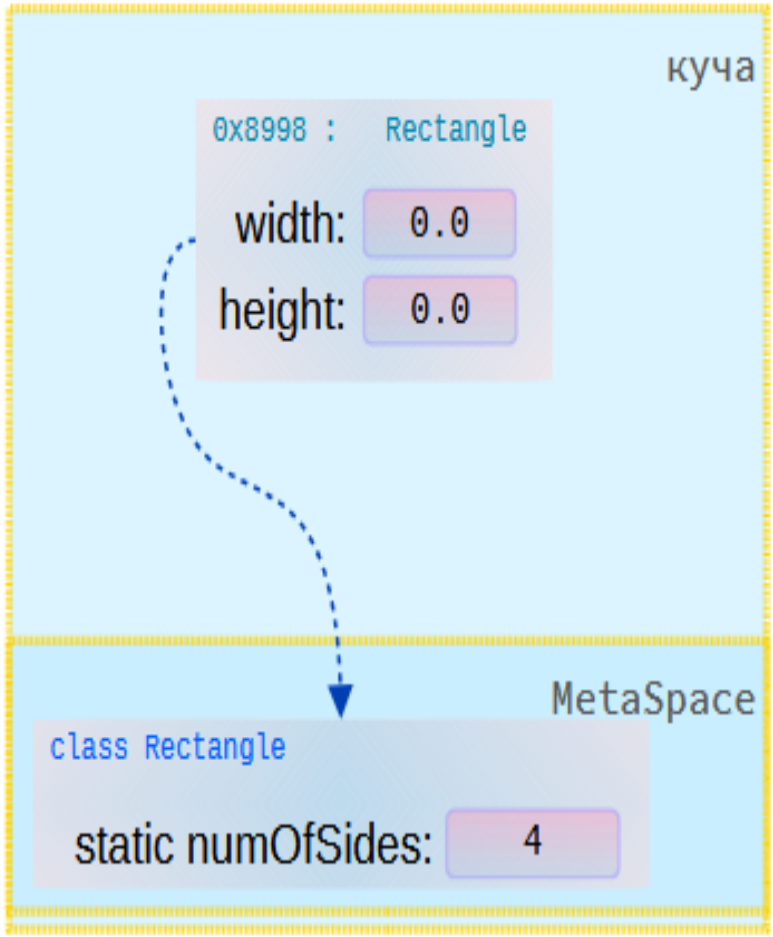


Статические поля

ИТМО

```
class Rectangle {
  double width;
  double height;
  static int numOfSides = 4;
}
```

```
Rectangle r1 = new Rectangle();
int x = r1.numOfSides; // 4
x = Rectangle.numOfSides;
```





```
class Rectangle {
   double this.width;
   double this.height;

   double area( Rectangle this ) {
      return this.width * this.height;
   }
}
```

Объекты в качестве полей

ИТМО

- Такое отношение между классами называется **композиция**

```
class Rectangle {
  double width;
  double height;
  Point corner;
}
```

```
class Point {
  double x;
  double y;
}
```

```
Point p1 = new Point();
p1.x = 30.0;
p1.y = 22.5;
Rectangle r1 = new Rectangle();
r1.width = 5.0;
r1.height = 7.0;
r1.corner = p1;
```



Задание: Напишите класс покемона

- Данные: имя, текущий уровень (начиная с 1), health, attackPower, healthInc, attackInc
- Методы: Конструктор (задает имя, и все статы), levelUp(увеличение статов по заданному инкременту),
  sayHello (вывести имя+ какую-нибудь фразу),
  attack(Pokemon& target) - изменение здоровья + лог боя в консоль
- Сами подумайте, что public, что private
- Доп1: Сделайте класс битвы, принимающий двух покемонов и крутящий поочерёдные удары в цикле.
- Здоровье, атаку и уровень лучше сделать приватными (чтобы только сам объект контролировал их). Тогда для их изменения нужны getter и setter функции: getHealth(), setHealth(). Такие функции очень часто используются в реальных проектах